

Quoridor（墙棋）游戏

一、项目背景

Quoridor，墙棋（又译“步步为营”），是一款1997年推出的棋盘游戏。在该游戏中，玩家以优先到达对侧棋盘底边为目标，轮到自己行动时，既可以移动自己的棋子，也可以通过放置墙壁，尝试阻挠对方棋子的行动。在游戏进行过程中，玩家永远不可以封死对方（包括自己）的全部通路，只能合理利用障碍尽可能延长对手的行进路线。

Quoridor的游戏规则十分简洁但有相当的游玩和策略深度，故选择复刻这一项目，并尝试实现了一个基于 Minimax 算法的初步对战 AI。

二、主要功能

- **游戏规则完整实现：**
 - **三种操作输入：** 移动 / 水平墙 / 垂直墙
 - **获胜判断**
 - **移动legal moves获取：** 一般移动和越过逻辑
 - **墙壁放置合法性判断：** 存在通路，交叉重叠
- **GUI：**
 - 基于 Tkinter 的图形界面，支持鼠标交互
 - 实时显示当前回合、剩余墙壁数量、操作反馈、虚影预览等
- **玩家模式：** 对于先（黑）后（白）手，皆可指定由玩家手动操作或AI操作
 - **人类玩家：** 通过点击棋盘移动或放置墙壁
 - **对战AI：** 基于 MiniMax 搜索 + Alpha-Beta 剪枝的 AI 对手

三、核心算法

1. BFS

- **墙壁放置合法性判断：**
- Quoridor规则的核心之一在于不可堵死对方（和自己）的行动通路。为了保证这一点，对于潜在的墙壁放置行动假设执行，通过用BFS判断双方是否皆存在剩余通路，确定合法性后回溯。
- **当前最小步数计算：**
- AI评估函数的计算需要考虑自己和对手的当前最小路径长度，用BFS实现。

2. 候选动作剪枝&排序

Quoridor行动的特殊之处在于除了基本的移动操作外还包括墙壁放置行动，后者在整张棋盘上存在海量可能，对于搜索算法来说过于复杂，必须加以限制。

- **移动行动：** 获取当前合法移动方式列表，根据移动后所得BFS最短路径升序排序。
- **墙壁行动（当剩余墙壁大于0时）：**
 - 获取对手当前位置&对手当前BFS路径下一步所在位置作为关注位置
 - 获取关注位置周围2格的范围内的合法放置方法
 - 根据对手最短路径增长&自己最短路径增长（负权）排序，截取前5个
 - *对自己重复该操作*

注： 真实情况下墙壁位置的考量可以具备相当大的远见和策略深度，此处运用启发式思想选取关注位置附近的个别操作是为搜索算法可行性做出的牺牲。将自己的位置同样列为关注会增加策略深度的小尝试，不过在实际运行中因评估策略的设计问题，基本不会被采用。

3. Minimax（含Alpha-Beta剪枝）

- AI 玩家在决策时，会模拟双方未来的走法（移动或放墙），构建一棵博弈树。在叶子节点处使用评估函数得到局面分数，并利用 **Alpha-Beta 剪枝** 剪掉不可能影响最终决策的分支，在有限时间内找到较优走法。
- 模型设置了最大时间3.0s和最大深度5，超时截断，避免过长的思考时间影响用户体验或造成更严重的问题。

4. 启发式评估函数

Quoridor的胜负只取决于谁先到达对方底线，因此，“达目标的最短路径长度”是最核心的胜负指标。此处选择直接以“对手最短路程”-“自己最短路程”作为评估函数，判断当前局势，为核心博弈过程提供参考。

四、 运行指南

1. 环境要求

- Python 3.8 或更高版本
- 这个项目**仅依赖 Python 标准库**，不需要安装额外的第三方包。

使用的标准库模块：

- `tkinter` — GUI 界面（Python 自带）
- `collections` — 数据结构工具
- `random` — 随机数生成
- `math` — 数学运算
- `time` — 计时功能

2、文件结构

项目目录/

- └─ quoridor_app.py # GUI 和程序入口
- └─ quoridor_core.py # 游戏核心逻辑
- └─ quoridor_players.py # 玩家定义（人类玩家和 AI）
- └─ README.md # 项目主说明文档

3、使用说明

- 运行

```
python quoridor_app.py
```

- 对局配置

启动后首先弹出「对局配置」对话框：

- 黑方（先手）：选择玩家类型（人类玩家 / Minimax AI）
- 白方（后手）：选择玩家类型（人类玩家 / Minimax AI）
- 点击「开始对局」进入游戏，点击「取消」则使用默认的人人对局模式
- 配置在开局后锁定，中途不可更改。

- 游戏界面

- 棋盘为 9×9 网格，黑方从**顶部**出发（目标：底部），白方从**底部**出发（目标：顶部）
- 顶部状态栏显示：当前回合玩家、双方剩余墙壁数量
- 底部状态栏显示：操作反馈和提示信息

- 基本操作

在界面下方的「输入模式」中选择当前操作类型：

模式	操作方式	说明
移动	点击棋盘上的格子	将当前棋子移动到目标格（高亮显示合法目标）
水平墙	点击墙槽位置	放置一面水平方向的墙壁
垂直墙	点击墙槽位置	放置一面垂直方向的墙壁

鼠标悬停时，会显示虚影预览：

-  绿色：该位置合法
-  红色：该位置非法（墙重叠 / 无通路 / 墙壁已用完）

- 重置/新对局

- 点击右上角「重置 / 新对局」按钮，重新弹出配置对话框
- 可选择新的玩家组合重新开始
- **游戏结束**
 - 当一方抵达目标行时，游戏自动结束
 - 弹出「**游戏结束**」对话框，显示获胜方
 - 点击「重置 / 新对局」可开始下一局

五、 AI使用及反思

GUI部分实现主要由AI实现。游戏主体部分和对战AI部分借助了AI搭建的框架，部分机械操作的实现由AI代为完成。核心算法部分接受了AI的指导（Minimax相关），最终由本人完成。BFS有关算法逻辑和评估函数的设计主要由本人独立实现。部分代码在完成后经过AI模块化整理和可读性增强。

所有的代码均经过人工review和检视。项目说明主要由本人编写，仅在游戏操作利用AI生成了部分内容。

反思：说实话我之前完全没有完整项目开发的经验，加上主题的选取主要来自于兴趣，对对弈AI实现的原理了解还相当有限。所以这次大作业的完成整体对AI的依赖还是比较高的，从框架搭建到实现思路都或多或少接受了AI的指导。但所有内容（除了GUI部分，这方面几乎完全不了解）最终完成前都已经经过从头到尾的检视并完全理解，核心逻辑部分也都是本人理解后手动写的。AI的设计部分还有许多优化空间，目前的智能程度相当有限。有关**评估函数**的设计：还有剩余墙体数等参数可以纳入考虑，应当采取AI对弈等方式得出最佳的参数设计，甚至可以考虑非线性模型。与此同时，可以引入随机数增强不确定性。此外，根本的模型选择也有更优选择，Quoridor的复杂度实际上一定程度超出了Minimax擅长的范围，但本人知识有限，本次大作业还是以通过实践掌握Minimax为主要目的，故没有考虑更加合适的算法。